

1. SCRUM

1.1 Definición

Scrum es un marco de trabajo ligero (no una metodología prescriptiva) diseñado para ayudar a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas frente a problemas complejos. Sus creadores insisten en que Scrum es deliberadamente incompleto: define solo las piezas necesarias para implementar la teoría del Scrum empírico, y deja que el resto de las prácticas, procesos y técnicas sean elegidas según el contexto de cada equipo.

1.2 Origen

Scrum surgió en 1986, cuando Hirotaka Takeuchi e Ikujiro Nonaka sentaron las bases del framework en su artículo “The New New Product Development Game”. Esta idea fue adaptada al desarrollo de software y presentada formalmente por Ken Schwaber y Jeff Sutherland en la conferencia OOPSLA de 1995. La primera versión de la Guía de Scrum se publicó en 2009, y el documento se ha actualizado desde entonces manteniendo como base el proceso empírico y los principios Lean. La versión vigente corresponde a noviembre de 2020, en la cual se simplificó el lenguaje, se redujo el carácter prescriptivo del marco y se pasó de hablar de equipos autoorganizados a equipos autogestionados.

1.3 Principios

Scrum se sostiene sobre tres pilares del empirismo:

- Transparencia: el proceso y el trabajo deben ser visibles tanto para quienes lo ejecutan como para quienes lo reciben.
- Inspección: los artefactos de Scrum y el avance hacia los objetivos acordados deben revisarse con frecuencia para detectar variaciones o problemas.
- Adaptación: cuando la inspección revela que un aspecto del proceso se desvía de límites aceptables, debe ajustarse cuanto antes.

A estos pilares se suman los cinco valores de Scrum: compromiso, foco, apertura, respeto y coraje, que orientan el comportamiento del equipo en su trabajo diario.

1.4 Roles

La Guía de Scrum 2020 define un único Equipo Scrum, pequeño y multifuncional, compuesto por:

- Product Owner: responsable de maximizar el valor del producto y de gestionar el Product Backlog.
- Scrum Master: responsable de establecer Scrum tal como se define en la Guía, facilita el trabajo del equipo y elimina impedimentos.

- Developers: responsables de crear cualquier aspecto de un Incremento utilizable en cada Sprint.

1.5 Eventos

- Sprint: evento contenedor de duración fija (normalmente entre una y cuatro semanas) dentro del cual ocurren todos los demás eventos.
- Sprint Planning: da inicio al Sprint, define qué se puede entregar y cómo se logrará el trabajo.
- Daily Scrum: reunión diaria de 15 minutos para que los Developers inspeccionen el avance hacia el Sprint Goal y ajusten el plan.
- Sprint Review: se inspecciona el resultado del Sprint con los interesados (stakeholders) y se adapta el Product Backlog si es necesario.
- Sprint Retrospective: el equipo revisa cómo fue el Sprint en cuanto a personas, procesos y herramientas, e identifica mejoras.

1.6 Artefactos

Cada artefacto de Scrum incluye un compromiso que aporta transparencia y foco:

Artefacto	Compromiso asociado
Product Backlog	Product Goal (objetivo del producto)
Sprint Backlog	Sprint Goal (objetivo del sprint)
Incremento	Definition of Done (definición de terminado)

El Product Backlog es la lista ordenada y emergente de todo lo que se necesita para mejorar el producto. El Sprint Backlog es el plan del Developers para el Sprint en curso. El Incremento es un escalón concreto hacia el objetivo del producto, y debe cumplir la Definición de Terminado para considerarse parte de él.

1.7 Ventajas

Según APMG International (organización de certificación en gestión de proyectos), entre los beneficios más citados de Scrum se encuentran:

- Mayor flexibilidad, ya que el enfoque iterativo e incremental permite a los equipos adaptarse a requisitos y prioridades cambiantes durante todo el desarrollo.
- Un tiempo de comercialización más rápido, porque los Incrementos “Terminados” se entregan en sprints cortos, adelantando la materialización de valor.
- Colaboración mejorada, favorecida por los eventos, artefactos y responsabilidades claras del marco.
- Mejor calidad del producto, gracias a la retroalimentación continua que permite identificar y resolver problemas de forma temprana.

- Mayor satisfacción del cliente, favorecida por un Product Owner dedicado a representar sus intereses.
- Mejora continua, impulsada por las retrospectivas regulares que permiten ajustar procesos y rendimiento.
- Mayor transparencia y previsibilidad del progreso, gracias a Sprints de cadencia regular y Sprint Goals acordados.

1.8 Desventajas

La misma fuente identifica desafíos frecuentes en la adopción de Scrum:

- Resistencia al cambio derivada de una comprensión deficiente del marco, con una curva de aprendizaje pronunciada.
- Dificultad de algunos equipos para sostener la colaboración y comunicación constantes que exige Scrum, percibiendo las reuniones frecuentes como una carga y confundiendo la inspección con microgestión.
- Fricción entre las responsabilidades individuales —especialmente las del Scrum Master— y las estructuras tradicionales de la organización.
- Riesgo de que los equipos se sobrecomprometan a medida que gana confianza, ya sea por presión interna o externa.
- Tendencia a sobreenfatizar los objetivos de corto plazo Sprint a Sprint, perdiendo de vista el objetivo general del producto.
- Dificultad para integrar el trabajo de múltiples equipos en esfuerzos grandes y complejos, sin un marco de escalamiento adicional.

2. EXTREME PROGRAMMING (XP)

2.1 Definición

Extreme Programming (XP) es una metodología ágil de desarrollo de software orientada a producir software de alta calidad de forma frecuente, a través de un conjunto de prácticas de ingeniería concretas más que de un marco de gestión de procesos. Se describe como una disciplina basada en cinco valores —simplicidad, comunicación, retroalimentación, coraje y respeto— que se llevan a la práctica mediante reglas específicas para cada etapa del desarrollo.

2.2 Origen

XP nació a mediados de los años noventa, cuando Kent Beck fue contratado como consultor en el proyecto Chrysler Comprehensive Compensation (C3) de Chrysler. Beck trabajó junto a Don Wells y Ron Jeffries, quienes documentaron y difundieron las prácticas del proceso. El equipo del proyecto C3 reportó mejoras sustanciales de productividad al adoptar estas prácticas frente al proceso previo. Kent Beck formalizó la metodología en su libro *Extreme Programming Explained* (Addison-Wesley, 2000), que se considera la obra de referencia del enfoque.

2.3 Principios / Valores

XP se sostiene sobre cinco valores centrales:

- Comunicación: favorecer el diálogo abierto y frecuente entre el equipo y los interesados, en lugar de depender de documentación extensa.
- Simplicidad: buscar siempre la solución más simple que funcione para el problema actual.
- Retroalimentación: obtener retroalimentación continua a través de pruebas, del cliente y del propio equipo.
- Coraje: tomar decisiones difíciles o hacer cambios necesarios, incluso en etapas avanzadas del proyecto.
- Respeto: valorar el aporte de cada integrante del equipo y fomentar un ambiente colaborativo.

2.4 Prácticas

XP se distingue de otras metodologías ágiles por definir un conjunto explícito de prácticas de ingeniería, agrupadas en cuatro categorías:

Planificación

- Uso de historias de usuario para capturar requisitos.
- El Planning Game o juego de planificación, donde cliente y equipo priorizan colaborativamente el trabajo.

- Entregas pequeñas y frecuentes (releases).
- División del proyecto en iteraciones cortas.

Diseño

- Diseño simple, evitando funcionalidad añadida antes de tiempo.
- Uso de una metáfora de sistema compartida para guiar el diseño.
- Refactorización constante del código sin alterar su comportamiento.

Codificación

- Cliente disponible de forma permanente para el equipo (on-site customer).
- Estándares de codificación acordados por el equipo.
- Programación en parejas (pair programming).
- Propiedad colectiva del código: cualquier persona del equipo puede modificar cualquier parte del código.
- Integración continua del código.
- Ritmo sostenible de trabajo, sin horas extra sistemáticas.

Pruebas

- Desarrollo guiado por pruebas (TDD): las pruebas unitarias se escriben antes que el código.
- Todo el código debe pasar las pruebas unitarias antes de integrarse.
- Las pruebas de aceptación se ejecutan con frecuencia y sus resultados se comunican al equipo.

2.5 Roles

XP define roles menos formalizados que Scrum, orientados a la práctica diaria:

- Cliente: define los requisitos mediante historias de usuario, prioriza el trabajo y está disponible para resolver dudas del equipo.
- Programador: diseña, implementa y prueba el software, trabajando habitualmente en parejas.
- Coach: guía al equipo en la correcta aplicación de las prácticas de XP.
- Tracker: hace seguimiento del progreso del equipo frente a las estimaciones realizadas.

2.6 Ciclo de vida / Artefactos

El desarrollo en XP se organiza en releases (entregas) compuestos por varias iteraciones cortas (habitualmente de una a tres semanas). Cada iteración parte de una selección de historias de usuario priorizadas por el cliente durante el Planning Game. El

avance del equipo se mide mediante la velocidad del proyecto (Project Velocity), que permite ajustar cuánto trabajo puede comprometerse en la siguiente iteración.

2.7 Ventajas

- Mejora de la calidad del código gracias a la combinación de TDD, refactorización constante y programación en parejas.
- Retroalimentación temprana y frecuente, tanto técnica (pruebas automatizadas) como del cliente (entregas pequeñas).
- Mayor capacidad de respuesta a cambios de requisitos, incluso en etapas avanzadas del proyecto.
- Transferencia de conocimiento dentro del equipo favorecida por la programación en parejas y la propiedad colectiva del código.

2.8 Desventajas

- Dificultad para escalar en equipos grandes o distribuidos geográficamente.
- Exige alta disciplina y compromiso constante del equipo con las prácticas técnicas (TDD, integración continua, pair programming).
- Depende fuertemente de la disponibilidad real del cliente, lo cual no siempre es viable en la práctica.
- Puede encontrar resistencia en organizaciones acostumbradas a procesos más tradicionales o documentados.

3. KANBAN

3.1 Definición

Kanban es un método de gestión del flujo de trabajo orientado al cambio evolutivo de los procesos existentes en una organización. A diferencia de Scrum y XP, Kanban no prescribe roles, eventos ni iteraciones de duración fija: su enfoque es visualizar el trabajo, limitar cuánto se realiza simultáneamente y gestionar el flujo de principio a fin, aplicándose sobre el proceso que el equipo ya tiene en lugar de reemplazarlo de inmediato.

3.2 Origen

Kanban tiene su origen en el sistema de producción de Toyota de la década de 1940-1960, donde se utilizaba como un método visual para gestionar el flujo de manufactura y reducir el desperdicio. David J. Anderson adaptó estos conceptos al trabajo de conocimiento (desarrollo de software y gestión de TI) a partir de 2004-2005, combinando ideas de W. Edwards Deming, Eli Goldratt, Peter Drucker y Taiichi Ohno. Anderson formalizó el método en su libro *Kanban: Successful Evolutionary Change for Your Technology Business* (2010), basado en su experiencia aplicando el enfoque en Microsoft.

3.3 Principios

El Kanban Method, según Kanban University, se basa en principios fundacionales orientados al cambio organizacional:

- Empezar con lo que se hace ahora: Kanban se superpone al proceso y roles actuales del equipo, sin exigir un rediseño inmediato.
- Acordar perseguir el cambio evolutivo: las mejoras se introducen de forma incremental, no mediante transformaciones radicales.
- Respetar inicialmente los roles, responsabilidades y cargos actuales: no se imponen nuevos roles desde el principio; el cambio surge del propio proceso.

3.4 Prácticas fundamentales

Kanban University define seis prácticas centrales que operacionalizan los principios anteriores:

- Visualizar el flujo de trabajo: representar el trabajo y su proceso, habitualmente mediante un tablero.
- Limitar el trabajo en curso (WIP): establecer un número máximo de elementos que pueden estar en progreso en cada etapa, para evitar la sobrecarga y exponer cuellos de botella.
- Gestionar el flujo: monitorear y optimizar el movimiento del trabajo a través del proceso, en lugar de gestionar tareas individuales.

- Hacer explícitas las políticas del proceso: definir con claridad las reglas bajo las cuales se mueve el trabajo (por ejemplo, criterios para pasar de una columna a otra).
- Implementar ciclos de retroalimentación: establecer puntos de revisión regulares para inspeccionar el flujo y el desempeño.
- Mejorar colaborativamente, evolucionar experimentalmente: usar modelos y el método científico para introducir cambios de forma colaborativa y basada en evidencia.

3.5 Roles

Kanban no prescribe roles formales, a diferencia de Scrum. El tercer principio fundamental establece explícitamente respetar los roles, responsabilidades y cargos que el equipo ya tiene. En implementaciones más maduras del método suelen aparecer roles de apoyo —como Service Delivery Manager o Service Request Manager—, pero no son obligatorios ni forman parte del núcleo mínimo del método.

3.6 Artefactos

- Tablero Kanban: representación visual del flujo de trabajo, dividido en columnas que representan las etapas del proceso (por ejemplo: Por hacer, En progreso, En revisión, Terminado).
- Tarjetas (kanban cards): representan cada elemento de trabajo individual que se mueve a través del tablero.
- Límites de WIP: restricciones numéricas asociadas a una o más columnas del tablero, que materializan la práctica de limitar el trabajo en curso.

3.7 Ventajas

- Flexibilidad para adoptarse sobre un proceso existente, sin requerir una reestructuración organizacional inmediata.
- Visualización clara del estado del trabajo y de los cuellos de botella, gracias al tablero y a los límites de WIP.
- Ausencia de iteraciones de duración fija, lo que permite una entrega continua en lugar de por lotes.
- Menor sobrecarga de proceso: no exige roles ni eventos obligatorios, lo que puede facilitar su adopción inicial.

3.8 Desventajas

- Al no definir roles ni cadencias fijas, puede requerir mayor disciplina del equipo para no perder el enfoque en la mejora continua.
- La falta de una estructura prescriptiva puede dificultar su implementación en equipos sin experiencia previa en métodos ágiles o Lean.

- El éxito depende en gran medida de establecer y respetar límites de WIP realistas; límites mal definidos reducen su efectividad.
- Al no incluir eventos de planificación o retrospectiva obligatorios, la reflexión y mejora del proceso puede diluirse si el equipo no los incorpora voluntariamente.

4. CRYSTAL

4.1 Definición

Crystal es una familia de metodologías ágiles centrada en las personas y su interacción, más que en procesos o herramientas. Su premisa central es que no existe una metodología universal: cada proyecto debe elegir un conjunto de prácticas adaptado a su tamaño de equipo y al nivel de criticidad del sistema que se construye. En lugar de prescribir procedimientos fijos, Crystal orienta a los equipos a lograr un conjunto de propiedades que las mejores prácticas han demostrado producir buenos resultados.

4.2 Origen

Crystal se originó en 1991, cuando IBM encargó a Alistair Cockburn diseñar una metodología para proyectos orientados a objetos. Cockburn entrevistó a distintos equipos de proyecto y observó que los equipos exitosos compartían prácticas comunes —comunicación cercana, buen ánimo del equipo, acceso a usuarios expertos— independientemente de la metodología formal que decían seguir. Aplicó estas ideas como consultor líder en el proyecto Orange de 1994 y documentó sus hallazgos en el libro *Surviving Object-Oriented Projects* (1997). Posteriormente formalizó la variante para equipos pequeños en *Crystal Clear: A Human-Powered Methodology for Small Teams* (2004), consolidando la familia completa de metodologías Crystal.

4.3 Principios

Crystal se basa en dos supuestos fundamentales, según Cockburn:

- Los equipos pueden optimizar su propio proceso de trabajo mientras lo ejecutan, en lugar de seguir un proceso fijo desde el inicio.
- Cada proyecto es único y dinámico, por lo que requiere un método adaptado a sus características particulares, no una plantilla universal.

A partir de estos supuestos, Cockburn identifica siete propiedades presentes en los equipos más exitosos. Las primeras tres se consideran obligatorias en Crystal Clear; las siguientes cuatro llevan al equipo más lejos en la “zona de seguridad” del proyecto:

1. Entrega frecuente de software funcionando y probado.
2. Comunicación osmótica (o comunicación cercana, en proyectos más grandes): la información fluye naturalmente entre los miembros del equipo que trabajan en un mismo espacio, de modo que la captan “por ósmosis” sin necesidad de que se les comunique formalmente.
3. Mejora reflexiva: el equipo se reúne periódicamente a reflexionar sobre su forma de trabajar y ajustarla.
4. Seguridad personal: los miembros del equipo pueden expresar desacuerdos o malas noticias sin temor.

5. Foco: claridad sobre las prioridades y el tiempo disponible para trabajar en ellas.
6. Fácil acceso a usuarios expertos.
7. Entorno técnico con pruebas automatizadas, gestión de configuración e integración frecuente.

4.4 Familia de metodologías Crystal

A diferencia de Scrum, XP y Kanban, Crystal no es una única metodología sino una familia codificada por colores, donde el color indica el “peso” (nivel de proceso y documentación) recomendado según el tamaño del equipo y la criticidad del sistema:

- Crystal Clear: equipos pequeños, colocalizados, de hasta 6-8 personas.
- Crystal Yellow: equipos de aproximadamente 10 a 20 personas.
- Crystal Orange: equipos de entre 20 y 50 personas.
- Crystal Red, Maroon y variantes adicionales (Blue, Violet): proyectos progresivamente más grandes y críticos, con mayor necesidad de coordinación formal.

Cuanto más grande o crítico es el proyecto, más “pesada” (con más documentación y control) debe ser la variante utilizada; los proyectos pequeños y de baja criticidad pueden operar con las variantes más ligeras.

4.5 Roles

Los roles varían según la variante de Crystal utilizada, pero en Crystal Clear —la más relevante para equipos pequeños— destacan:

- Patrocinador (Sponsor): define la dirección del proyecto y provee los recursos.
- Diseñador principal (Lead Designer): encabeza el diseño técnico de la solución.
- Desarrolladores: entre dos y siete personas adicionales que implementan el sistema, trabajando en un mismo espacio o en salas adyacentes.
- Usuario experto: provee retroalimentación frecuente y cercana al equipo de desarrollo.

4.6 Prácticas / Artefactos

Crystal no prescribe artefactos ni prácticas técnicas obligatorias más allá de las propiedades ya descritas, pero recomienda técnicas que ayudan a lograrlas:

- Talleres de reflexión (reflection workshops): sesiones breves y periódicas donde el equipo documenta en un lugar visible qué funcionó y qué debe ajustarse.
- Radiadores de información: pizarras o gráficos visibles en el espacio de trabajo que comunican el estado del proyecto sin necesidad de reuniones formales.
- Entregas incrementales cada uno o dos meses (trimestralmente como máximo), que sirven como punto de control natural del avance.

4.7 Ventajas

- Alta adaptabilidad: cada equipo puede elegir las técnicas concretas que le funcionen, siempre que logren las propiedades deseadas.
- Pone énfasis en las personas y su comunicación por encima de la documentación extensa, lo cual puede acelerar la resolución de problemas en equipos pequeños.
- Es liviana en procesos para proyectos pequeños, evitando la sobrecarga de documentación innecesaria.
- Escala de forma explícita: la familia ofrece variantes para proyectos de distinto tamaño y criticidad, en lugar de forzar una única forma de trabajar.

4.8 Desventajas

- Al no prescribir procedimientos concretos, requiere que el equipo tenga la madurez suficiente para definir sus propias prácticas.
- Existe menos documentación pública y menos herramientas o certificaciones ampliamente adoptadas en comparación con Scrum o Kanban, lo cual dificulta encontrar referencias exhaustivas sobre todas las variantes de la familia.
- La comunicación osmótica depende de que el equipo esté físicamente colocalizado, lo cual es una limitación relevante en contextos de trabajo remoto o distribuido.
- La elección de la variante correcta (Clear, Yellow, Orange, etc.) exige un criterio inicial sobre tamaño y criticidad que puede no ser evidente al comienzo de un proyecto académico.

5. COMPARACIÓN

5.1 Criterios de comparación

1. Adecuación a un desarrollador individual.
2. Rigidez de roles.
3. Manejo del cambio de requisitos.
4. Cadencia de entrega.
5. Nivel de documentación/proceso exigido.
6. Enfoque principal.
7. Dependencia de disponibilidad del cliente/usuario experto.

5.2 Tabla comparativa

Criterio	Scrum	XP	Kanban	Crystal
1. Adecuación a un desarrollador individual	Baja. Los tres roles recaerían en una sola persona, perdiendo el valor de la separación de responsabilidades.	Media. TDD, refactorización e integración continua son aplicables; pair programming y el rol de Cliente no lo son de forma literal.	Alta. No exige roles ni tamaño mínimo de equipo.	Baja-media. Crystal Clear está diseñada para 2-8 personas; con una sola persona no se ajusta literalmente, aunque varias propiedades individuales sí aplican.
2. Rigidez de roles	Alta. Tres roles definidos (Product Owner, Scrum Master, Developers).	Media. Roles menos formalizados (Cliente, Programador, Coach, Tracker).	Ninguna. No prescribe roles; respeta los existentes.	Media. Roles definidos en Crystal Clear (Sponsor, Lead Designer, Desarrolladores, Usuario experto), no obligatorios en todas las variantes.
3. Manejo del cambio de requisitos	Alto dentro de cada Sprint cerrado; los cambios se incorporan en el siguiente Sprint.	Alto; el valor de coraje y prácticas como diseño simple y refactorización asumen cambios frecuentes, incluso tardíos.	Muy alto; al no haber iteraciones fijas, el trabajo puede reordenarse en cualquier momento.	Alto; los principios asumen que el equipo optimiza su proceso sobre la marcha.
4. Cadencia de entrega	Fija, por Sprints (1-4 semanas).	Fija, por iteraciones cortas (1-3 semanas) dentro de releases.	Continua (flujo), sin iteraciones de duración fija.	Periódica flexible: entregas cada 1-2 meses (trimestral como máximo).
5. Nivel de documentación/proceso exigido	Medio. Definido por eventos y artefactos, sin plantillas de documentación prescritas.	Bajo-medio. Énfasis en código y pruebas automatizadas más que en documentación.	Bajo. Solo exige visualizar el flujo y hacer explícitas las políticas del proceso.	Variable según la variante; bajo en Crystal Clear, mayor en variantes para proyectos más grandes/críticos.
6. Enfoque principal	Gestión del proceso y del trabajo en Sprints.	Prácticas de ingeniería de software (calidad técnica del código).	Gestión y visualización del flujo de trabajo.	Propiedades centradas en las personas y su comunicación.
7. Dependencia de	Media. El Product	Alta. Una de sus	Baja. No define esta	Alta en las

disponibilidad del cliente/usuario experto	Owner representa al cliente, sin exigir su presencia constante.	prácticas centrales es el "cliente disponible" de forma permanente.	dependencia de forma explícita.	propiedades avanzadas ("fácil acceso a usuarios expertos"), no obligatoria en el mínimo de Crystal Clear.
--	---	---	---------------------------------	---

6. CONCLUSIÓN

La investigación permitió caracterizar cuatro metodologías ágiles con enfoques distintos: Scrum y XP se apoyan fuertemente en la interacción entre roles o entre personas (gestión de proceso en el primer caso, prácticas de ingeniería en el segundo); Kanban se centra en visualizar y gestionar el flujo de trabajo sin prescribir roles ni cadencias fijas; y Crystal prioriza propiedades centradas en las personas, con una familia de variantes que escala según el tamaño y la criticidad del proyecto.

Con base en la comparación de la sección 5, se selecciona Kanban como metodología de trabajo del proyecto. Los criterios más determinantes fueron:

- Ausencia de roles obligatorios y de un tamaño mínimo de equipo, directamente relevante dado que el desarrollador es una sola persona.
- Alta capacidad de manejo del cambio de requisitos, adecuada para un proyecto académico cuyo alcance y funcionalidades aún no están confirmados.
- Cadencia de entrega continua, más compatible con la disponibilidad irregular de un estudiante que los compromisos de Sprint fijo de Scrum o las iteraciones de XP.
- Baja dependencia de disponibilidad constante de un cliente o usuario experto, lo cual es favorable al no contar con un Pentester real confirmado como usuario disponible.

Esta selección reconoce una limitación: Kanban es la metodología con menor exigencia de documentación de las cuatro estudiadas (criterio 5 de la comparación). Esto no representa un problema para el desarrollo en sí mismo, pero implica que la trazabilidad y documentación exigidas por la asignatura deberán sostenerse mediante los artefactos propios del proyecto académico —historias de usuario, requisitos, diagramas—, y no mediante artefactos que Kanban no provee.

7. REFERENCIAS

Scrum

- Schwaber, K. & Sutherland, J. (2020). The Scrum Guide. Versión oficial en español: <https://www.scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf>
- Scrum.org. Sitio oficial del framework Scrum: <https://www.scrum.org>
- Craddock, A. (2025). Los 10 Principales Beneficios de Scrum y Desafíos Frecuentes. APMG International: <https://apmg-international.com/es/article/top-10-beneficios-de-scrum-y-desafios-frecuentes>

Extreme Programming (XP)

- Wells, D. Extreme Programming: A Gentle Introduction. Réplica académica (Universidad de Génova): <https://person.dibris.unige.it/reggio-gianna/ISII03WWW/ExtremeProgramming/rules.html>
- Beck, K. (2000). Extreme Programming Explained: Embrace Change. Addison-Wesley.
- Python Wiki. ExtremeProgramming (archivo histórico): <https://wiki.python.org/moin/ExtremeProgramming.html>

Kanban

- Kanban University. The Official Guide to The Kanban Method: <https://kanban.university>
- Anderson, D. J. (2010). Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press.
- Atlassian. What is Kanban in Project Management: <https://www.atlassian.com/agile/kanban>

Crystal

- Cockburn, A. (2004). Crystal Clear: A Human-Powered Methodology for Small Teams. Addison-Wesley. Capítulo de muestra: https://ptgmedia.pearsoncmg.com/images/0201699478/samplechapter/cockburn_ch02.pdf
- PMI (Project Management Institute). Definición de Comunicación Osmótica.
- Atlassian. The Crystal family of Agile methodologies: <https://www.atlassian.com/agile>